

Computer Organization & Architecture (BE04000251)



Unit-2 Register Transfer and Micro-operations

**Vrunda Dave
Assistant Professor
IT Department
V.V.P. Engineering College, Rajkot**

Basic computer data types

- Binary information is stored in **memory** or **processor registers**.
- Registers either contain **data** or **control information**.
 - Data are numbers and other binary-coded information
 - Control information is a bit or a group of bits used to specify the sequence of command signals needed for manipulation of the data in other registers.
- The data types(given by user through keyboard) found in registers of digital computers may be classified as follows:
 1. Numbers used in arithmetic computations (Eg.0.....9)
 2. Letters of the alphabet used in data preprocessing (Eg.A.....Z,a....z)
 3. Other symbols used for specific purpose (Eg. @, #)
- The binary number system is the most natural system to use in a digital computer.
- **Why Binary number system.....???????**
 - Registers are made up of flip-flops and flip-flops are two state devices that can only store 1's and 0's.
- Basic unit of information is '**binary digit**' or '**bit**'.

Outline

- Register Transfer language
- Bus and Memory Transfers (Tree-State Bus Buffers, Memory Transfer)
- Arithmetic MicroOperations
- Logic Micro-Operations
- Shift Micro-Operations
- Arithmetic logical shift unit

Register transfer language

Register transfer language

- A digital system is an interconnection of digital hardware modules.
- Modules are constructed from such digital components as registers, decoders and control logic.
- Digital modules are best defined by the **registers they contain** and the **operations that are performed on the data stored in them**.
- Registers are sequence of flip-flops where each flip-flop stores 1 bit of information.
- The basic operations performed on contents of register are called **micro operations(Arithmetic, Shift and Logical micro operations)**.

Register transfer language

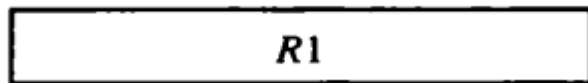
- The internal hardware organization of a digital computer is best defined by specifying:
 1. The set of registers it contains and their function.
 2. The sequence of microoperations performed on the binary information stored in the registers.
 3. The control that initiates the sequence of microoperations.
- It is more convenient to adopt a suitable symbology to describe sequence of transfer between registers than using explanation of every operation in words.
- The symbolic notation used to describe the micro operation transfers among registers is called a **register transfer language(register transfer + language)**.
- A register transfer language is a system for expressing in symbolic form the micro operation sequences among the registers of a digital module.

Register transfer

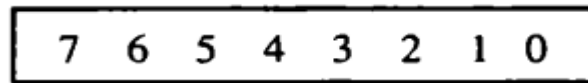
- Computer Registers are designated by capital letters(sometimes followed by numerals) to denote the function of the register.
- **Example:**
- The register that holds an address for the memory unit is usually called a **Memory Address Register (MAR)**.
- Other registers are PC (Program Counter), IR (Instruction Register) and R1 (Processor Register).
- The individual flip-flops in an n-bit register are numbered in sequence from 0 through n-1, starting from 0 in the rightmost position and increasing the numbers towards the left.
- A 16-bit register is partitioned into two part. Bits 0 through 7 are assigned the symbol L (**Low Byte**) and bits 8 through 15 are assigned the symbol H (**High Byte**).
- Example: PC(H) and PC(L)

Register transfer

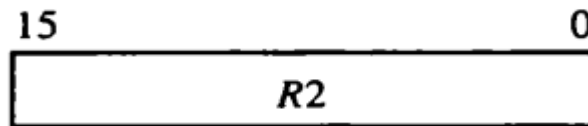
Figure 4-1 Block diagram of register.



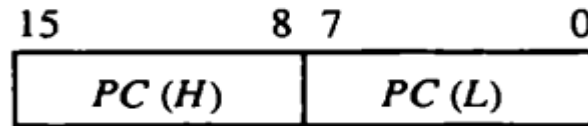
(a) Register *R*



(b) Showing individual bits



(c) Numbering of bits



(d) Divided into two parts

Register transfer

- Transfer means transferring data from one register to another register.
- Information transfer from one register to another is designated in symbolic form by means of a **replacement operator**(replaces content of one register into another one).
- **R2 <-R1** : It denotes a transfer of the content of register R1 into register R2 and the content of the source register R1 does not change after the transfer.

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two micro operations	$R2 \leftarrow R1, R1 \leftarrow R2$

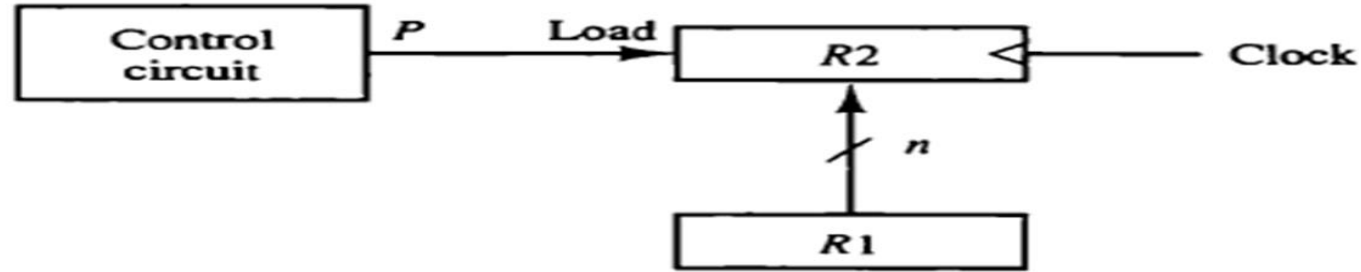
Fig. Basic Symbols for Register Transfers

Register transfer with control function

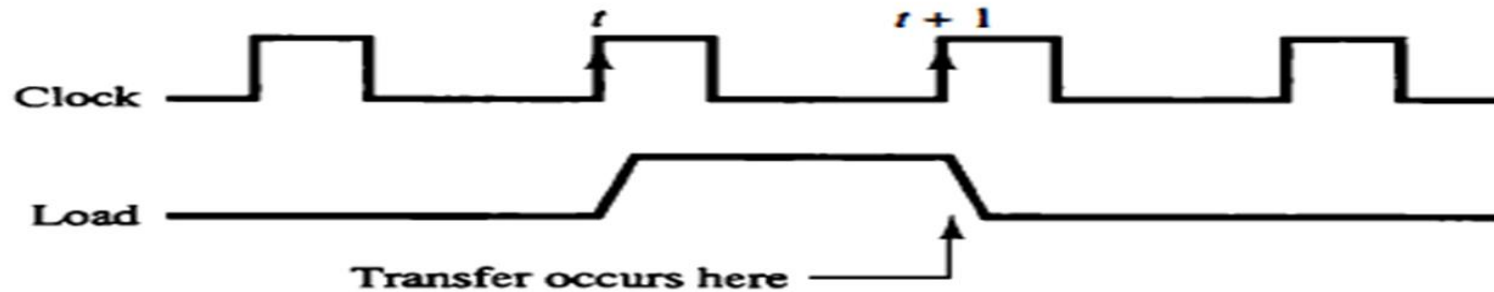
- We can perform that operation only when certain condition is true. For that we are taking a variable P.
- P is control variable that is generated by control unit.
- If $(P=1)$ then $(R2 \leftarrow R1)$: Register Transfer with Control Function
- $P: R2 \rightarrow R1$ (**Control Function** that contain boolean value(1 or 0))
- Load inputs load the corresponding data to the destination register.
- Whenever a clock pulse is applied then automatically control unit generated the control signal. so, the n bits of Register R1 are transferred to R2.
- At time t, we have initiated transfer of R1 content but for transfer we require **one clock pulse**.
- When the clock pulse is over then the entire n bits of R1 are transferred to R2.
- At $t+1$, transfer operation successfully completed.

Register transfer with control function

Figure 4-2 Transfer from $R1$ to $R2$ when $P = 1$.



(a) Block diagram



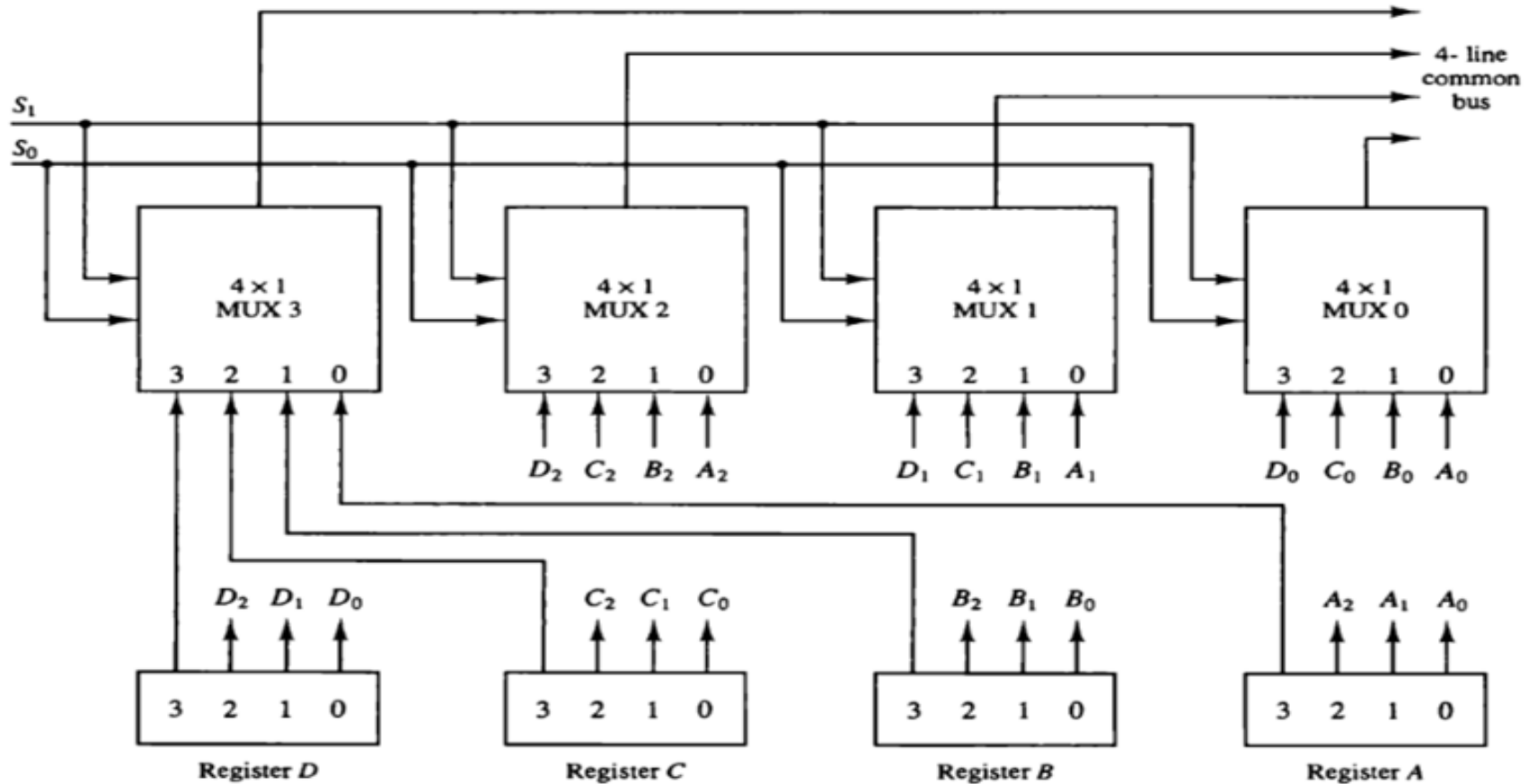
(b) Timing diagram

Bus and Memory transfers

Bus system

- Digital computer has many registers, and paths must be provided to transfer information from one register to another.
- Number of wires will be more if separate lines are used between each register & all other register in a system.
- A more efficient scheme for transferring information between registers in a multiple-register configuration is a common bus system.
- Bus Structure consists of a set of common lines, one for each bit of a register, through which information is transferred.
- Bus System can be constructed using two methods:
 1. Bus System for four registers using MUX
 2. Using three state buffer
- Control signals determine which register is selected by the bus during each particular register transfer

Bus system for four registers using MUX



Bus system for four registers using MUX

- How the different types of register can communicate with each other using common bus.
- Bus consist of 4x1 MUX each having four data inputs, 0 through 3, the two selection lines S1 and S0 .
- Diagram shows that the bits in the same significant position in each register are connected to the data inputs of one multiplexer to form one line of the bus.
- For example, output 1 of register A is connected to input 0 of MUX 2 because this input is labeled A1 . Thus MUX 0 multiplexes the four 0 bits of the registers, MUX 1 multiplexes the four 1 bits of the registers, and similarly for the other two bits.
- The two selection lines S1 and S0 are connected to the selection inputs of all four multiplexers. The selection lines choose the four bits of one register and transfer them into the four-line common bus.

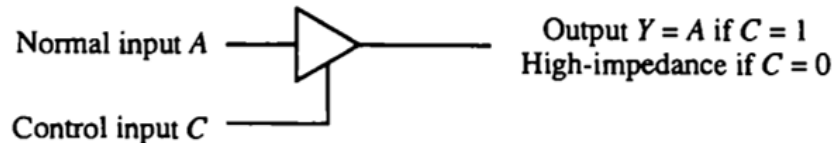
Bus system for four registers using MUX

- When $S_1S_0 = 00$, the 0 data inputs of all four multiplexers are selected and applied to the outputs that form the bus. This causes the bus lines to receive the content of register A since the outputs of this register are connected to the 0 data inputs of the multiplexers.
- Similarly, register B is selected if $S_1S_0 = 01$, and so on.
- Data of particular register will be loaded to bus and we can transfer it to ALU or to any register.
- In general, a bus system will multiplex k registers of n bits each to produce an n-line common bus. The number of multiplexers needed to construct the bus is equal to n , the number of bits in each register. The size of each multiplexer must be k x 1 since it multiplexes k data lines.

S_1	S_0	Register Selected
0	0	A
0	1	B
1	0	C
1	1	D

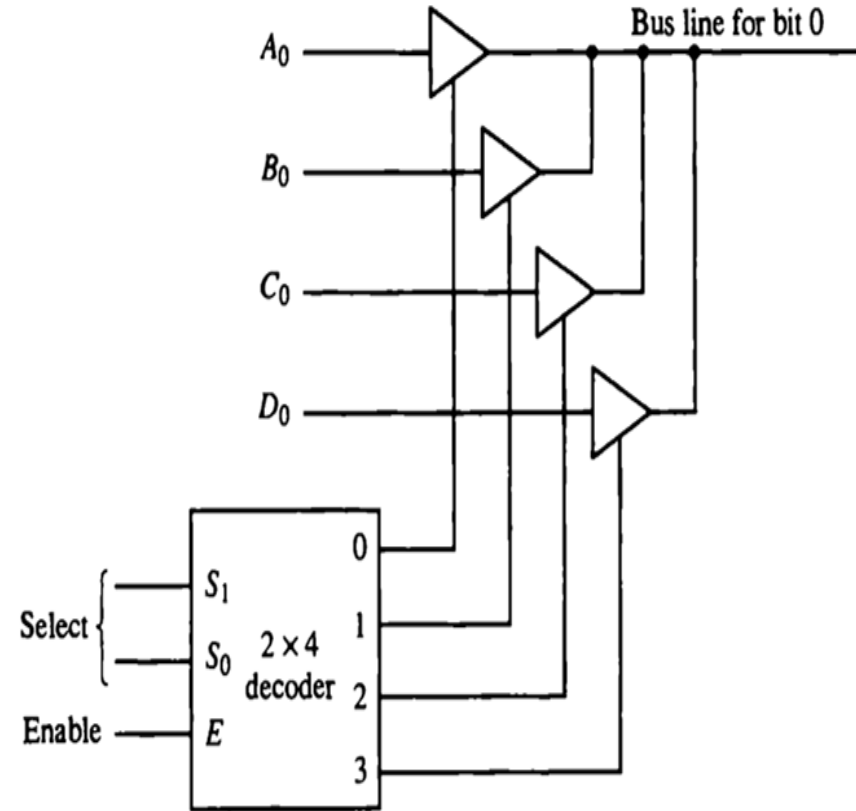
Bus system with three state buffers

- A bus system can be constructed with **three-state gates**. Two of the states are signals equivalent to logic 1 and 0 as in a conventional gate. The third state is a **high-impedance state** i.e it behaves like **an open circuit**, which means that the output is disconnected and does not have a logic significance.
- The one most commonly used in the design of a bus system is the buffer gate. The control input determines the output state. **When the control input is equal to 1, the output is enabled and the gate behaves like any conventional buffer, with the output equal to the normal input.**
- **When the control input is 0, the output is disabled and the gate goes to a high-impedance state, regardless of the value in the normal input.**



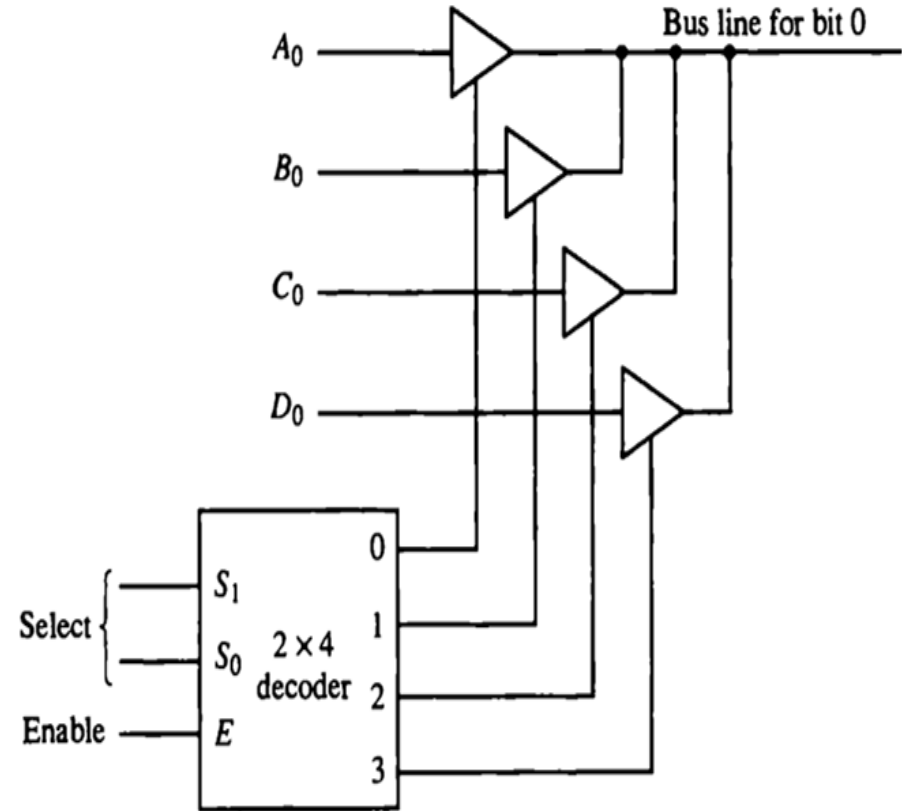
Bus system with three state buffers

- The outputs of four buffers are connected together to form a single bus line.
- The control inputs to the buffers determine which of the four normal inputs will communicate with the bus line. One buffer may be in the active state at any given time.
- The connected buffers must be controlled so that only one of the three-state buffer has access to the bus line while all other buffers are maintained in a high impedance state.



Bus system with three state buffers

- When the enable input of the decoder is 0, all of its four outputs are 0, and the bus line is in a high-impedance state because all four buffers are disabled.
- When the enable input is active, one of the three-state buffers will be active, depending on the binary value in the select inputs of the decoder.



Memory Transfer Operation

- Mainly two memory transfer operations are there: 1) Memory Read 2) Memory Write.
- Memory word will be symbolized by the letter M. The particular memory word among the many available is selected by the memory address during the transfer. It is necessary to specify the address of M when writing memory transfer operations.
- Consider a memory unit that receives the address from a register, called the address register (AR) . The data are transferred to another register, called the data register (DR).
- The transfer of information from a memory word to the outside environment is called a **read operation**.
 - It can be stated as, **Read: DR <-M [AR]**
 - This causes a transfer of information into DR from the memory word M selected by the address in AR .

Memory Transfer Operation

- The transfer of new information to be stored into the memory is called a **write operation**.
 - It can be stated as, **Write: M [AR] <-R1**
 - This causes a transfer of information from R1 into the memory word M selected by the address in AR.

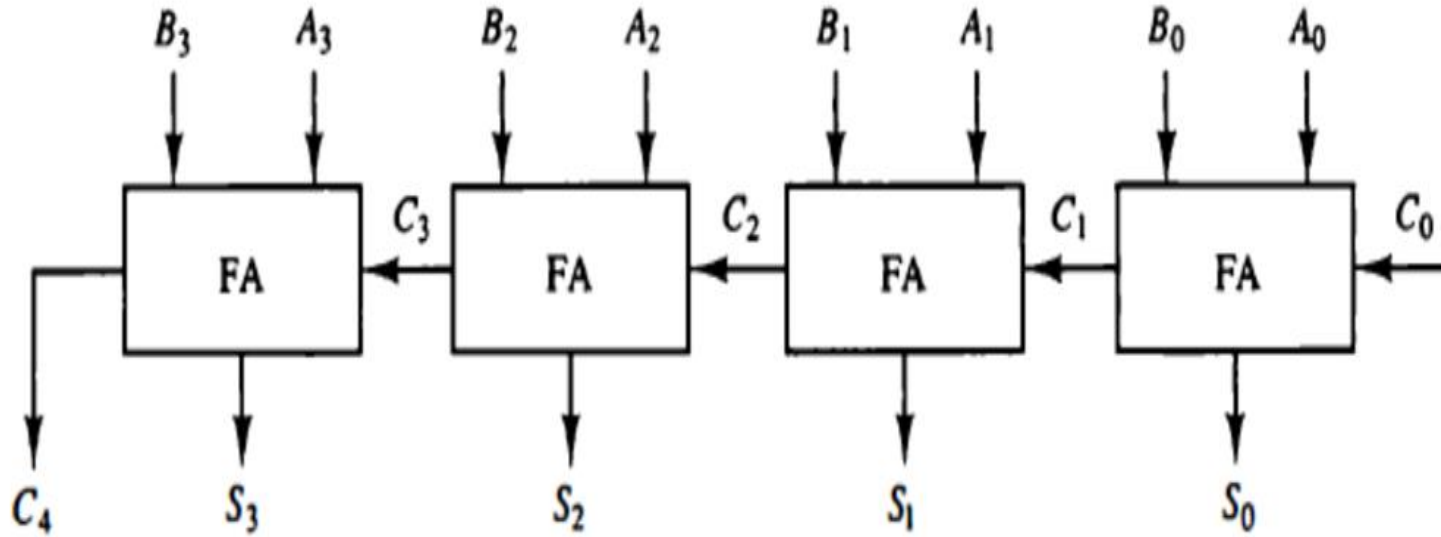
Arithmetic Micro Operation

Arithmetic Micro Operation

- Arithmetic micro-operations are mainly useful to perform arithmetic operations on contents of registers.
- Arithmetic micro-operations are performed by arithmetic and logical unit (ALU). The basic arithmetic micro operations are addition, subtraction, increment, decrement, and complement.

Symbolic Designation	Description
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow \overline{R2}$	Complement the contents of R2 (1's complement)
$R2 \leftarrow \overline{\overline{R2}} + 1$	2's complement the contents of R2
$R3 \leftarrow R1 + \overline{\overline{R2}} + 1$	R1 plus the 2's complement of R2 (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by one
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by one

Binary Adder



Binary Adder

- The digital circuit that forms the arithmetic sum of two bits and a previous carry is called a **full-adder**. The digital circuit that generates the arithmetic sum of two binary numbers of any length is called a **binary adder**.
- The binary adder is constructed with full-adder circuits connected in cascade, with the output carry from one full-adder connected to the input carry of the next full-adder. The carries are connected in a chain through the full-adders. The input carry to the binary adder is C_0 and the output carry is C_4 . The S outputs of the full-adders generate the required sum bits.
- In general, an n-bit binary adder requires n full-adders. The output carry from each full-adder is connected to the input carry of the next-high-order full-adder. The n data bits for the A inputs come from one register (such as R1), and the n data bits for the B inputs come from another register (such as R2). The sum can be transferred to a third register or to one of the source registers (R1 or R2), replacing its previous content.

Binary Adder- Subtractor

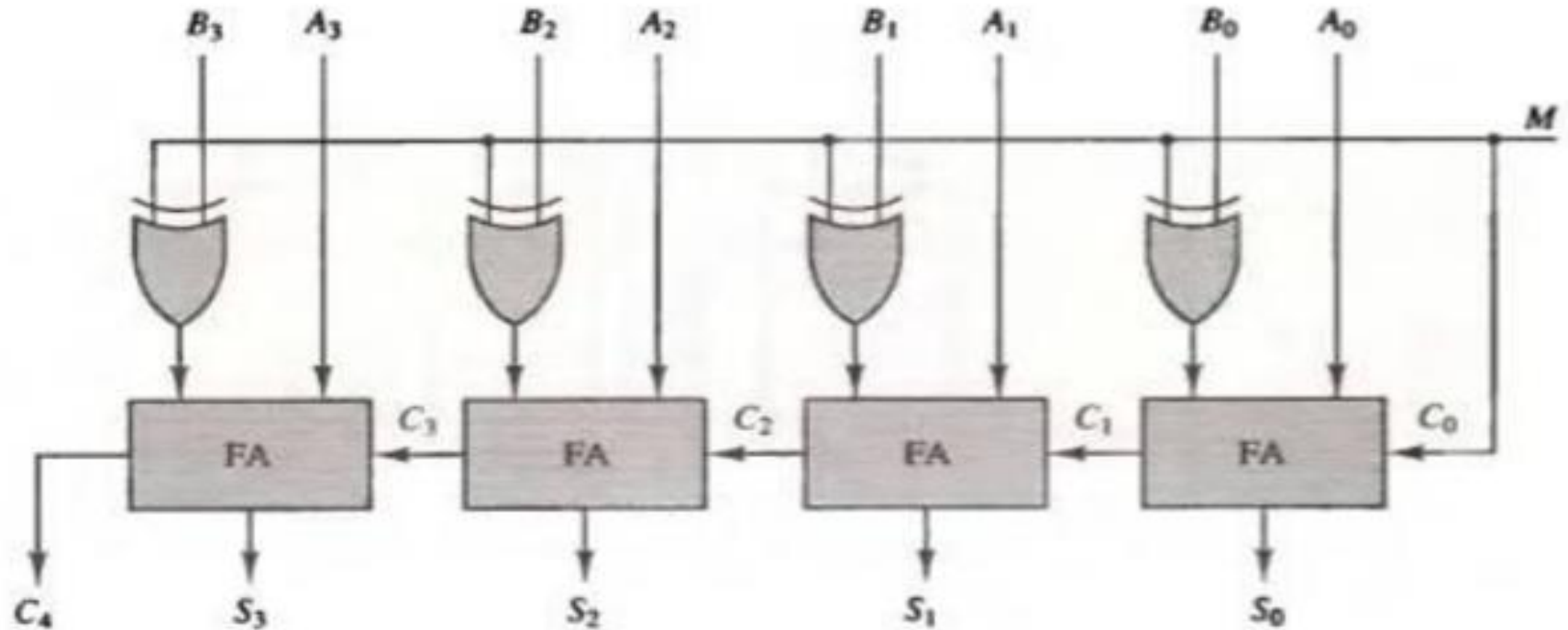


Figure 4-7 4-bit adder-subtractor.

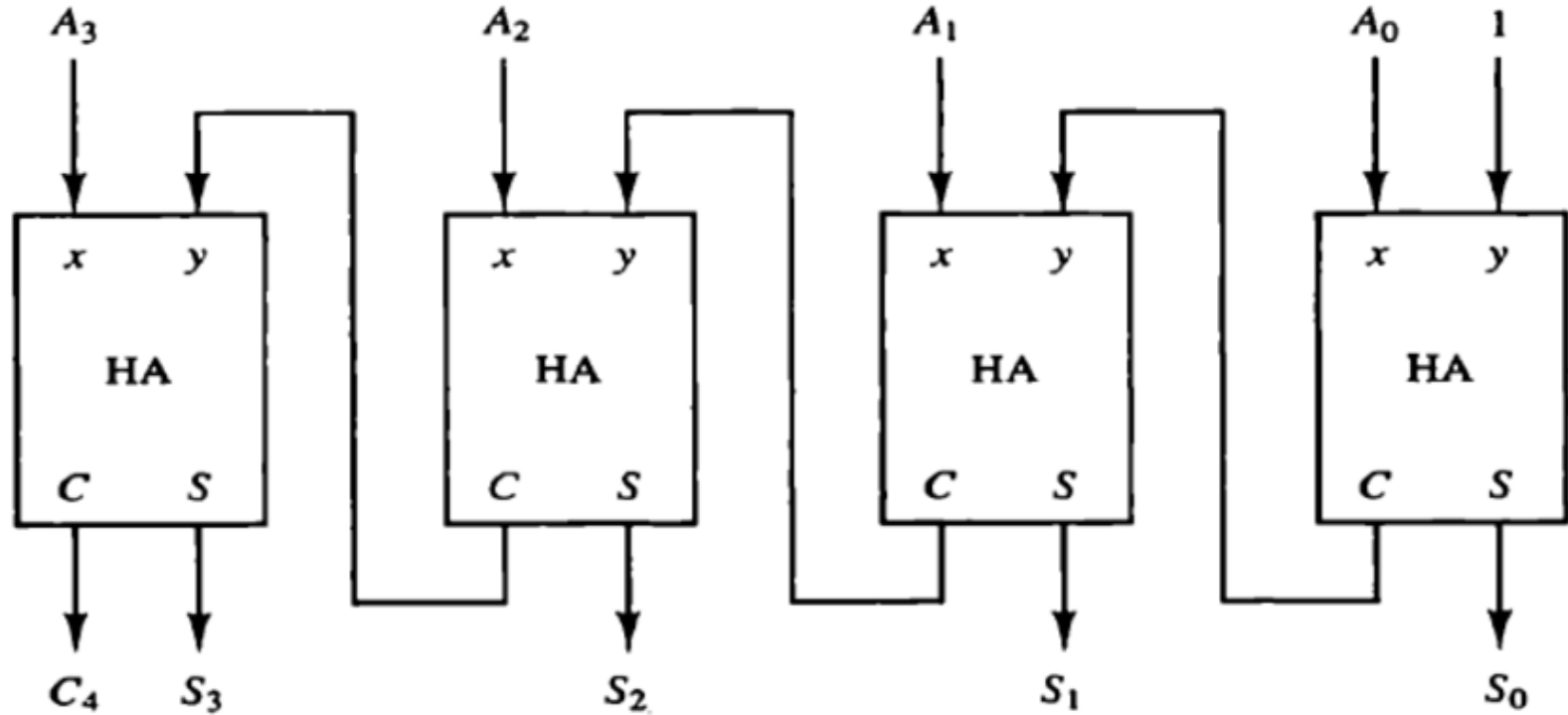
Binary Adder-Subtractor

- Subtraction can be performed using 2's complement.
- $A-B$ means $A+2$'s complement of B .
- **How can we perform 2's complement.....????**
 - Taking 1's complement and adding one to the LSB pair of bits.
- 1's complement can be implemented with inverters and one can be added to the sum through the input carry.

Binary Adder-Subtractor

- The addition and subtraction operations can be combined into one common circuit by including an **exclusive-OR gate** with each full-adder.
- The **mode input M controls the operation**.
- When $M = 0$ the circuit is an adder and when $M = 1$ the circuit becomes a subtractor.
- Each exclusive-OR gate receives input M and one of the inputs of B.
- When $M = 0$, we have $B \oplus 0 = B$.
- The full adders receive the value of B, the input carry is 0 and the circuit performs A plus B.
- When $M = 1$, we have $B \oplus 1 = B'$ and $C_0 = 1$.
- The B inputs are complemented and 1 is added through input carry.
- The subtraction $A - B$ can be done by taking the 2's complement of B and adding it to A ($A + B' + 1$).

Binary Incrementer

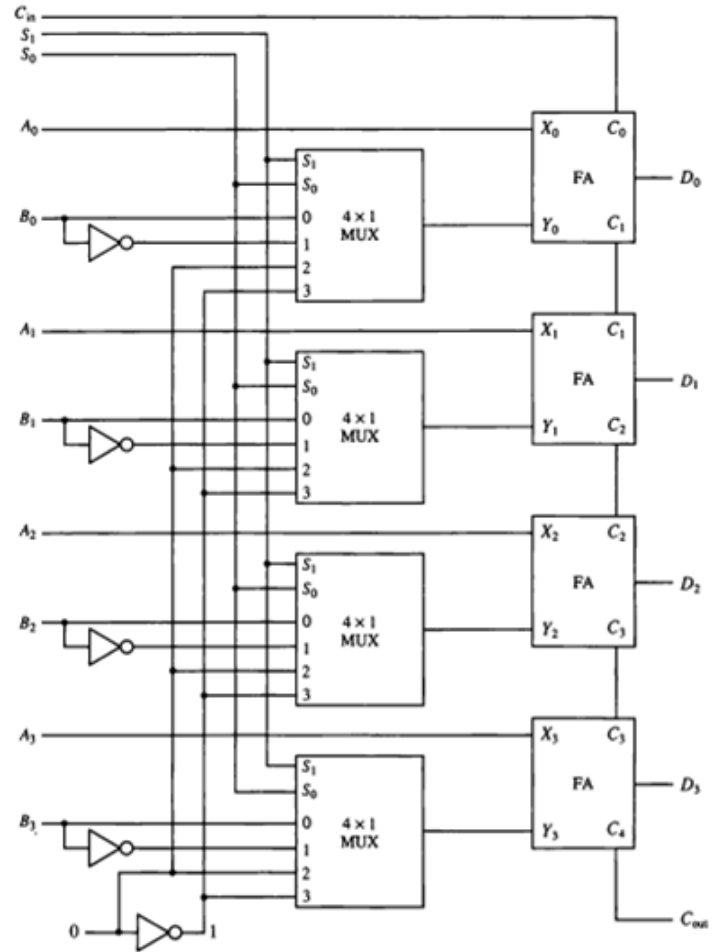


Binary Incrementer

- One of the inputs to the least significant half-adder (HA) is connected to logic-1 and the other input is connected to the least significant bit of the number to be incremented.
- The output carry from one half-adder is connected to one of the inputs of the next higher order half adder.
- The circuit receives the four bits from A0 through A3 , adds one to it, and generates the incremented output in S0 through S3 , The output carry C4 will be 1 only after incrementing binary 1111. This also causes outputs S0 through S3 to go to 0.
- In general, an n-bit Binary Incrementer by extending the diagram to include n half-adders. The least significant bit must have one input connected to logic-1. The other inputs receive the number to be incremented or the carry from the previous stage.

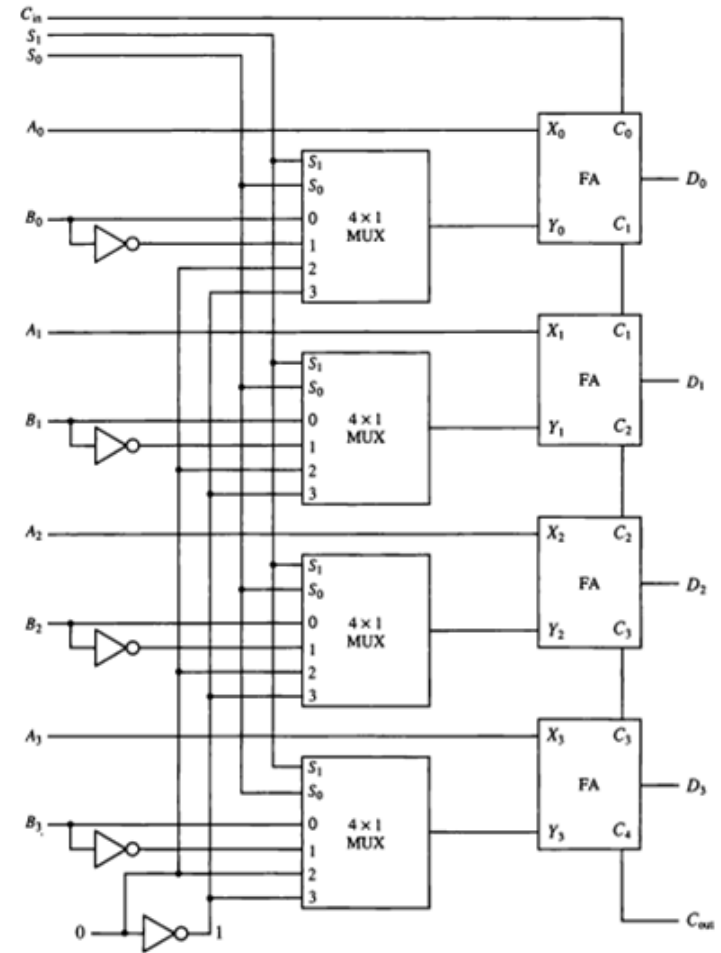
Arithmetic Circuit

- By controlling the data inputs to the adder, it is possible to obtain different types of arithmetic operations.
- It has four full-adder circuits that constitute the 4-bit adder and four multiplexers for choosing different operations. There are two 4-bit inputs A and B and a 4-bit output D.
- The four inputs from A go directly to the X inputs of the binary adder. Each of the four inputs from B is connected to the data inputs of the multiplexers. The multiplexer's data inputs also receive the complement of B. The other two data inputs are connected to logic-0 and logic-1.



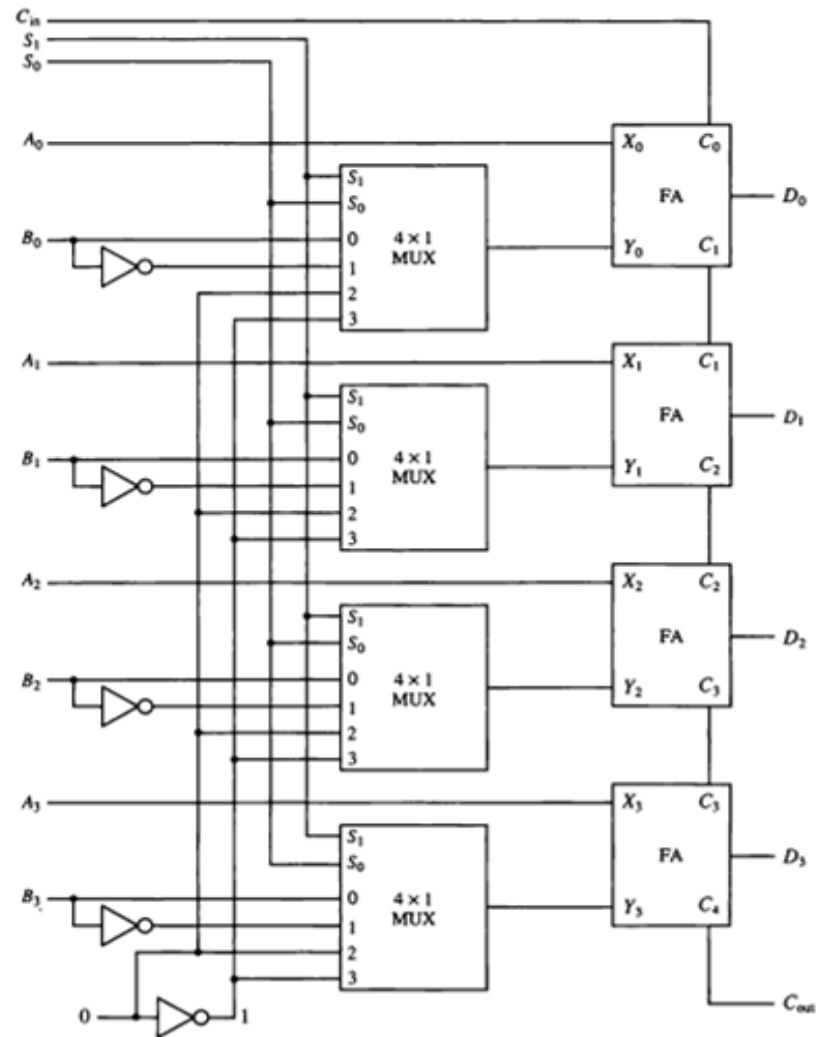
Arithmetic Circuit

- The four multiplexers are controlled by two selection inputs, S_1 and S_0 . Input carry C_{in} goes to the carry input of the full adder (FA) in the least significant position. The other carries are connected from one stage to the next.
- The output of the binary adder is calculated from the following arithmetic equation: $D = A + Y + C_{in}$



Arithmetic Circuit

Select			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S_1	S_0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A



Arithmetic Circuit

- **Addition:** When $S_1S_0 = 00$, the value of B is applied to the Y inputs of the adder. If $C_{in} = 0$, the output $D = A+B$. If $C_{in} = 1$, output $D = A + B + 1$.
- **Subtraction:** When $S_1S_0 = 01$, the complement of B is applied to the Y inputs of the adder. If $C_{in} = 1$, then $D = A + B + 1$. (i.e. 2's complement subtraction of B , which is equivalent to a subtraction of $A - B$). When $C_{in} = 0$, then $D = A + B$. This is equivalent to subtract with borrow, i.e. $A - B - 1$.

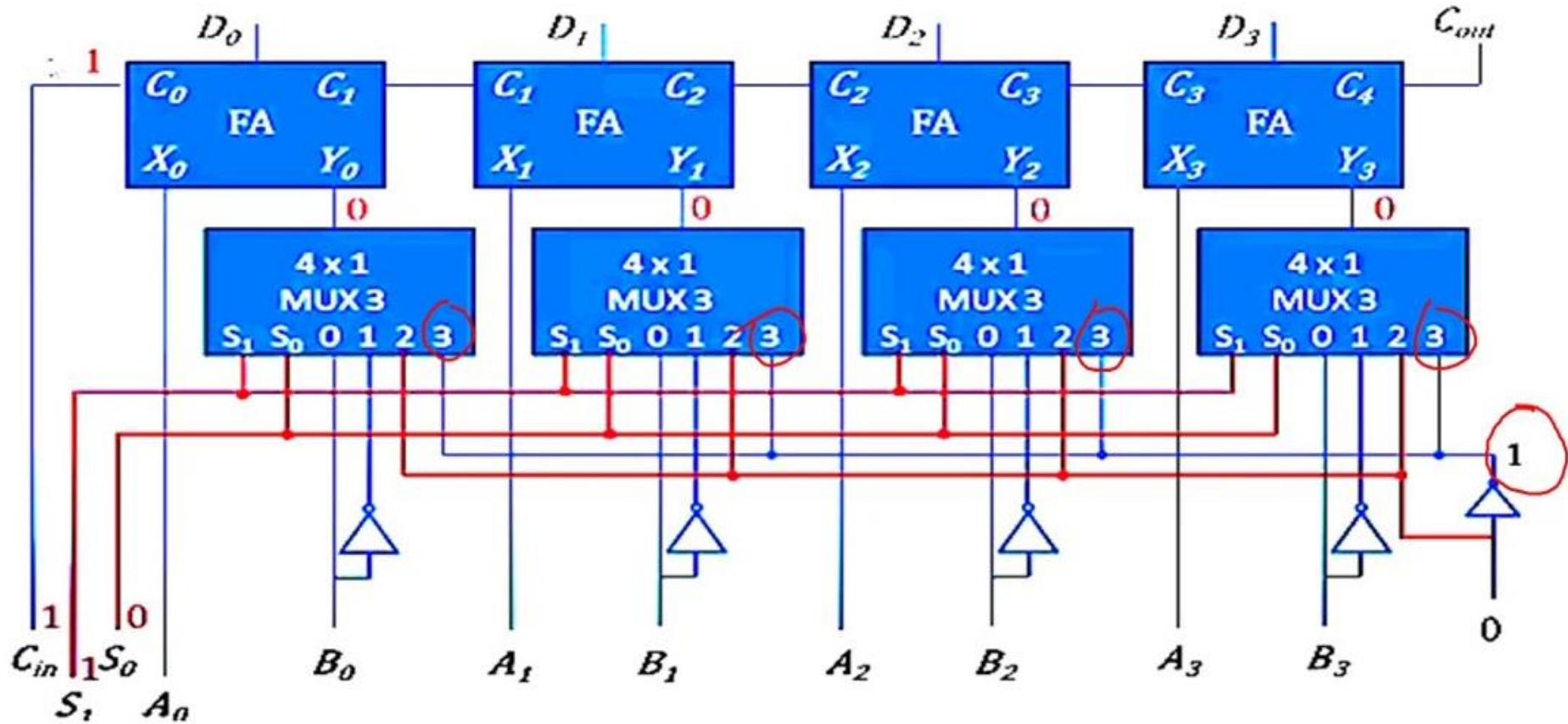
Select			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S_1	S_0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Arithmetic Circuit

- **Increment:** When $S_1S_0 = 10$, the inputs from B are neglected, and instead, all 0's are inserted into the Y inputs. The output becomes $D = A + 0 + C_{in}$. When $C_{in} = 0$, then $D = A$ and when $C_{in} = 1$, then $D = A + 1$.
- **Decrement:** When $S_1S_0 = 11$, all 1's are inserted into the Y inputs of the adder to produce the decrement operation $D = A - 1$ when $C_{in} = 0$. This is because a number with all 1's is equal to the 2's complement of 1. Adding a number A to the 2's complement of 1 produces $F = A + 2$'s complement of 1 = $A - 1$. When $C_{in} = 1$, then $D = A - 1 + 1 = A$, which causes a direct transfer from input A to output D.

Select			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S_1	S_0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Arithmetic Circuit (Landscape view)



Logic Micro operation

Logic Micro operation

- Logic micro operation specifies **binary operations on the strings of bits in registers**.
- These operations consider **each bit of the register separately** and treat them as binary variables.
- **Example:** the **exclusive-OR** microoperation with the contents of two registers R1 and R2 is symbolized by the statement
- **$R1 \leftarrow R1 \oplus R2$**
- It specifies that the microoperation is executed on individual bits of the registers.
- **Numerical Example:**

1010	Content of R1
<u>1100</u>	Content of R2
0110	Content of R1 after $P = 1$
- Some special symbols will be adopted for the logic microoperations OR($\vee$) , AND($\wedge$) etc.

Logic Micro operation

- There are 16 different logic micro operations that can be performed with two binary variables.
- The 16 logic operations are derived from these boolean functions by replacing variable x by the binary content of register A and variable y by the binary content of register B.

TABLE 4-5 Truth Tables for 16 Functions of Two Variables

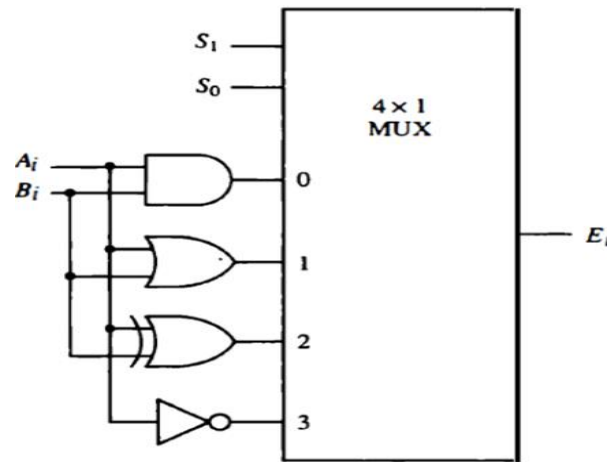
<i>x</i>	<i>y</i>	<i>F</i> ₀	<i>F</i> ₁	<i>F</i> ₂	<i>F</i> ₃	<i>F</i> ₄	<i>F</i> ₅	<i>F</i> ₆	<i>F</i> ₇	<i>F</i> ₈	<i>F</i> ₉	<i>F</i> ₁₀	<i>F</i> ₁₁	<i>F</i> ₁₂	<i>F</i> ₁₃	<i>F</i> ₁₄	<i>F</i> ₁₅
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

TABLE 4-6 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer <i>A</i>
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer <i>B</i>
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement <i>B</i>
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement <i>A</i>
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Hardware Implementation of Logic Circuit

- There are 16 microoperations but most computers use only four- AND, OR, XOR and complement- from which all others can be derived.
- This circuit generates the four basic logic micro operations.
- It consists of four gates and a multiplexer.
- The two selection lines of the multiplexer selects one of the four logic operations available at one time.
- The circuit shows one stage for bit “i” but for logic circuit with n bits, the circuit should be repeated n times for $i=0,1,2,\dots,n-1$.
- The Selection variables are applied to all stages.



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

Application of Logic Micro operation

- Logic microoperations are very useful for manipulating individual bits or a portion of a word stored in a register.
- They can be used to change bit values, delete a group of bits, or insert new bit values into a register.
- Examples show how the bits of one register(designated by A) are manipulated by logic microoperations as a function of the bits of another register(designated by B).

Application of Logic Micro operation

- **Selective-set operation:** Sets to 1 the bits in register A where there are corresponding 1's in register B. It does not affect bit positions that have 0's in B.

➤ **For example :**

Register A	1010	before
Register B	1100	(logic operand)
Register A	1110	after

➤ The OR micro operation can be used to selectively-set bits of a register.

- **Selective-complement operation:** Complements bits in A where there are corresponding 1's in register B. It does not affect bit positions that have 0's in B.

➤ **For example :**

Register A	1010	before
Register B	1100	(logic operand)
Register A	0110	after

➤ The Exclusive-OR micro operation can be used to selectively complement bits of a register.

Application of Logic Micro operation

- **Selective-clear operation:** Clears to 0 the bits in A only where there are corresponding 1's in register B.

➤ **For example :**

Register A	1010	before
Register B	1100	(logic operand)
Register A	0010	after

➤ The corresponding logic micro operation is $A \wedge B'$

- **Mask operation:** It is similar to the selective-clear operation except that the bits of A are cleared only where there are corresponding 0's in B.

➤ **For example :**

Register A	1010	before
Register B	1100	(logic operand)
Register A	1000	after masking

➤ The corresponding logic micro operation is AND.

Application of Logic Micro operation

- **Insert operation:** It inserts a new value into a group of bits. This is done by first **masking the bits** and then **ORing them** with the required value.

➤ **For example :** Insert 1001 in place of 0110.

and then insert the new value:

Register A	0110 1010	before
Register B	<u>0000 1111</u>	(mask)
Register A	0000 1010	after masking

Register A	0000 1010	before
Register B	<u>1001 0000</u>	(insert)
Register A	1001 1010	after insertion.

- The mask operation is an AND micro operation and the insert operation is an OR micro operation.
- **Clear operation:** It compares the words in A and B and produces an all 0's result, if the two numbers are equal.

Register A	1010
Register B	<u>1010</u>
Register A	0000

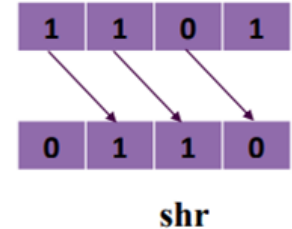
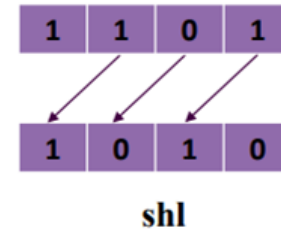
➤ **For example :**

- The corresponding logic micro operation is XOR operation.

Shift Micro operation

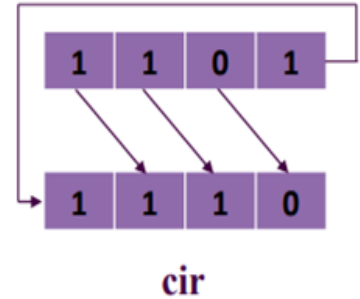
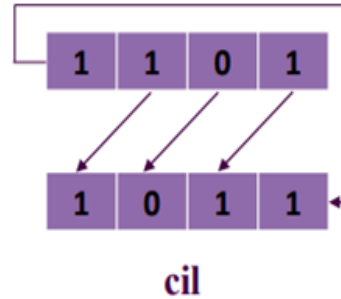
Shift Micro operation

- Shift micro operations are used for serial transfer of data. The contents of a register can be shifted to the left or the right.
 - **Which bit does it transfer???** → Depends upon the type of shift operation.
 - During a shift-left operation the serial input transfers a bit into the rightmost position.
 - During a shift-right operation the serial input transfer a bit into the leftmost position.
 - **Logical Shift:** A logical shift is one that **transfers 0 through the serial input**. The symbols **shl** and **shr** for logical shift-left and shift-right microoperations.
- **R ->shl R** :Shift-left register R
 - **R ->shr R** :Shift-right register R
 - It specifies a 1-bit shift to the left of the content of register R1 and a 1-bit shift to the right of the content of register R2.
 - Major problem is MSB(in shl) or LSB(in shr) will be discarded.



Shift Micro operation

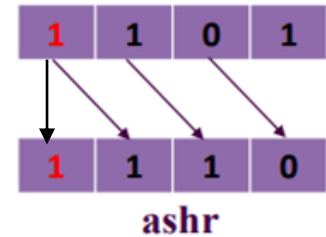
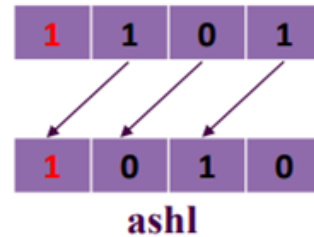
- **Circular shift (also known as a rotate operation):**
 - Overcomes the problem of logical shift.
 - It circulates the bits of the register around the two ends without loss of information. This is accomplished by **connecting the serial output of the shift register to its serial input**.
 - The symbols **cil** and **cir** is used.
 - **R ->cil R** : Circular shift-left register R
 - **R ->cir R** : Circular shift-right register R



Shift Micro operation

- **Arithmetic shift:**

- An arithmetic shift is a micro operation that **shifts a signed binary number to the left or right**.
- Arithmetic shifts **must leave the sign bit unchanged**. The leftmost bit in a register holds the sign bit, and the remaining bits hold the number.
- The sign bit is 0 for positive and 1 for negative. Negative numbers are in 2's complement form.
- **R -> ashl R** : Arithmetic shift-left R
- **R -> ashr R** : Arithmetic shift-right R
- $V_s = a_0 \oplus a_1$
- if $V_s = 0$ then no overflow and if $V_s = 1$ then there is overflow and we have replace the sign bit.
- V_s must be transferred to the overflow flip-flop.



4-bit Combinational Circuit Shifter

4-bit Combinational Circuit Shifter

- The 4-bit shifter has four data inputs A_0 through A_3 and four data outputs H_0 through H_3 .
- There are two serial inputs, one for shift(I_L) left and one for shift right(I_R).
- When the selection input $S=0$ the input data are shifted right and serial input transfers I_R .
- When the selection input $S=1$ the input data are shifted left and serial input transfers I_L .
- Functional table shows which input goes to each output after the shift.

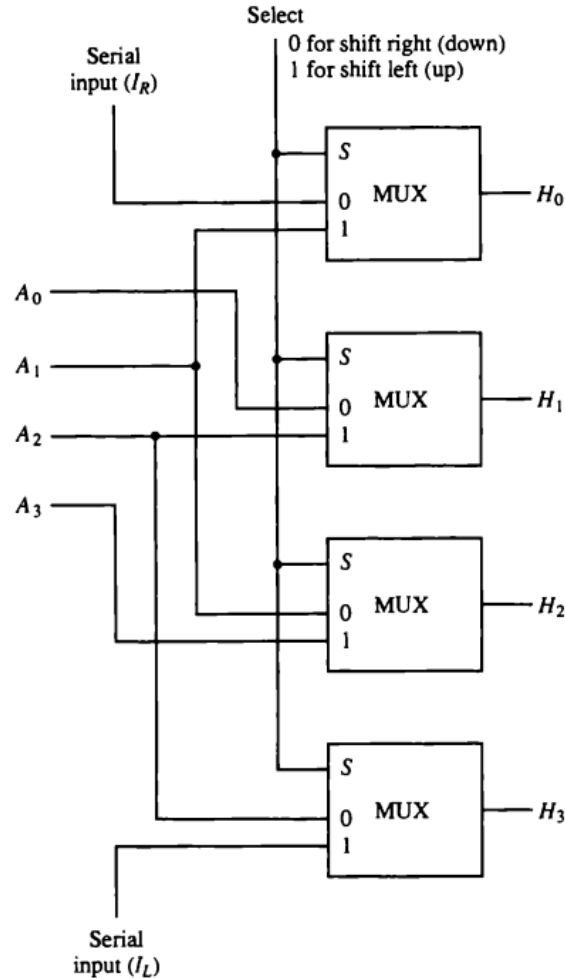
4-bit Combinational Circuit Shifter

Shift Right (S=0)

A0	A1	A2	A3
I_R	A0	A1	A2

Shift Left (S=1)

A0	A1	A2	A3
A1	A2	A3	I_L



Function table

Select	Output			
	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

Arithmetic Logic Shift Unit

Arithmetic Logic Shift Unit(ALU)

- Computer systems employ a number of storage registers connected to a common operational unit called an arithmetic logic unit.
- This circuit(ALU) performs arithmetic microoperations, logic microoperations and shift microoperations
- To perform a micro operation, the contents of specified registers are placed in the inputs of the common ALU. The ALU performs an operation and the result of the operation is then transferred to a destination register.
- The subscript i designates a typical stage. Inputs A_i and B_i are applied to both the arithmetic and logic units.

Arithmetic Logic Shift Unit(ALU)

- Here we have 4X1 multiplexer so then it has two selection line.
- If $S_3S_4=00$ then it performs arithmetic microoperations.
- If $S_3S_4=01$ then it performs logic microoperations.
- If $S_3S_4=10$ then it performs shift right microoperations.
- If $S_3S_4=11$ then it performs shift left microoperations.
- S_1S_0 controls the arithmetic and logic circuit.
- A_i is the one bit of register A and B_i is the one bit of register B.
- **Arithmetic Circuit:**
- Logic performs the corresponding operation based on selection line and produces D_i as the output and given to the input 0 to the multiplexer.
- C_i is the input carry and c_{i+1} is the output carry that is given to the next stage of the arithmetic circuit.
- If we want to perform data on n bits then we have to repeat the stage for n times.

Arithmetic Logic Shift Unit(ALU)

- **Logic Circuit:**
- Logic performs the corresponding operation based on selection line and produces E_i as the output and given to the input 1 to the multiplexer.
- Four basic logic microoperations: AND, OR, XOR and Complement.
- **Shift Circuit:**
- A_{i-1} represents a bit which is useful in order to perform shift right operation.
- A_{i+1} represents a bit which is useful in order to perform shift left operation.

Operation select

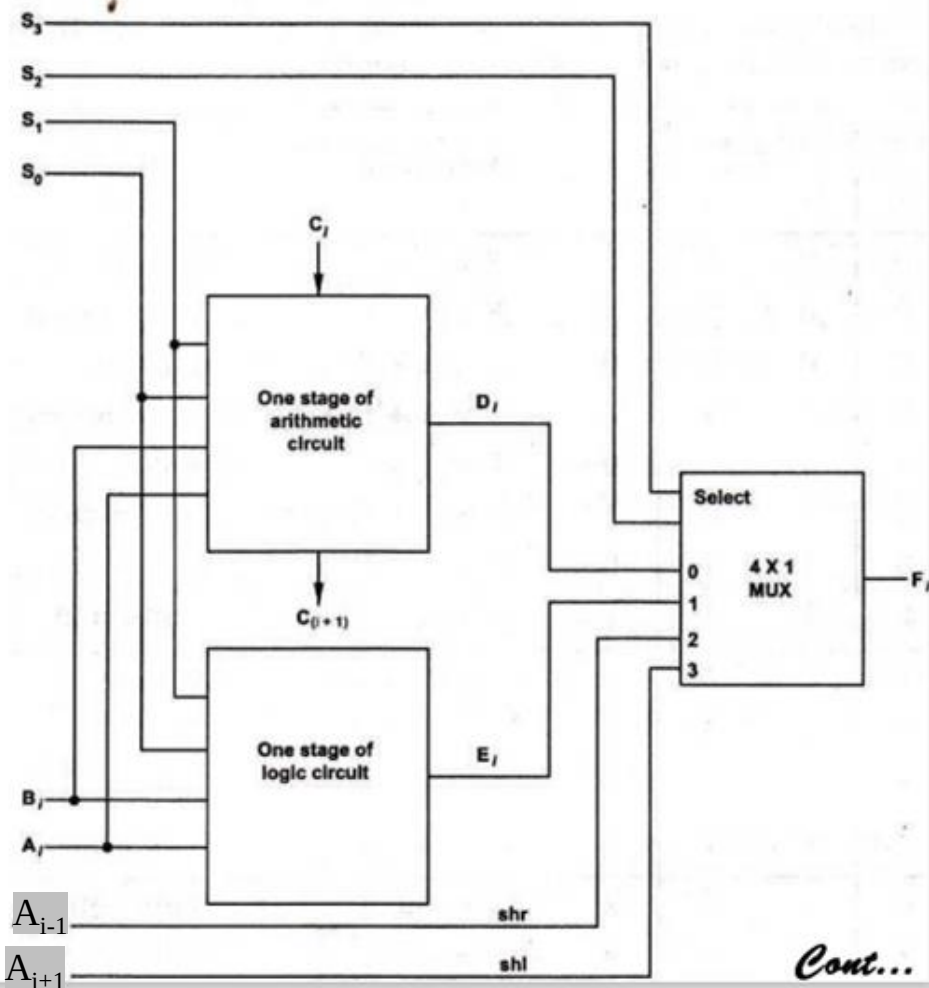
S_3	S_2	S_1	S_0	C_{in}	Operation	Function
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	$\times$	$F = A \wedge B$	AND
0	1	0	1	$\times$	$F = A \vee B$	OR
0	1	1	0	$\times$	$F = A \oplus B$	XOR
0	1	1	1	$\times$	$F = \overline{A}$	Complement A
1	0	$\times$	$\times$	$\times$	$F = \text{shr } A$	Shift right A into F
1	1	$\times$	$\times$	$\times$	$F = \text{shl } A$	Shift left A into F

Shift Right

A3	A2	A1	A0
0	A3	A2	A1

Shift Left

A3	A2	A1	A0
A2	A1	A0	0



Cont...

GTU Questions

Q.1 Explain signed representation of integer in computer. Write (-12) in binary with 8 – bits using following representations (i) Signed magnitude (ii) Signed 1's complement (iii) Signed 2's complement.

Q.2 Draw block diagram for 4-bit adder – subtractor.

Q.3 Show different ways to represent fixed-point positive integers including zero.

Q.4 Explain any four Arithmetic operations carried out by ALU.

Q.5 Apply the combinational circuits to design a 4-bit adder/subtractor circuit which performs subtraction using 2's complement.

Q.6 Draw the block diagram of 4-bit combinational circuit shifter.

Q.7 Construct diagram of common bus system of four 4-bits registers with diagram.

Q.8 Explain three state buffers.

Q.9 Define RTL. Give an example of register transfer of data through accumulator.

Q.10 Discuss 4-bit binary adder with neat diagram.

Q.11 Discuss three state bus buffers with neat diagram.

Q.12 Perform $A - B$ (subtract) operation for the following numbers using signed magnitude number format. (Write necessary assumptions if required) $A = + 11$ and $B = - 6$.

Q.13 Write the difference(s) between arithmetic shift left and logical shift left instruction. Support your answer with proper illustration.

Q.14 What is binary and decimal equivalent of F8 hexadecimal value?

GTU Questions

Q.15 Write Steps for two n digit numbers subtraction in base r.

Q.16 What is arithmetic micro-operation?

Q.17 State and Explain any seven logic micro-operation.

Q.18 Explain register transfer using block diagram and timing diagram.

Q.19 Explain how $(r-1)$'s complement is calculated. Calculate 9's complement of 546700.

Q.20 Define RTL. Give block diagram and timing diagram of transfer of R1 to R2 when $P=1$.

Q.21 Design a simple arithmetic circuit which should implement the following operations: Assume A and B are 3 bit registers. Add : $A+B$, Add with Carry: $A+B+1$, Subtract: $A+B'$, Subtract with Borrow: $A+B'+1$, Increment A: $A+1$, Decrement A: $A-1$, Transfer A: A.

Q.22 Explain how addition and subtraction of signed data is performed if a computer system uses signed magnitude representation.

GTU Questions

Q.1 Explain signed representation of integer in computer. Write (-12) in binary with 8 – bits using following representations (i) Signed magnitude (ii) Signed 1's complement (iii) Signed 2's complement.

- Only one way to represent $+12(00001100)$, there are three different ways to represent (-12) with eight bits
 - In signed-magnitude representation 1 0001100
 - In signed-1's complement representation 1 1110011
 - In signed-2's complement representation 1 1110100

GTU Questions

Q.3 Show different ways to represent fixed-point positive integers including zero.

Fixed-point positive integers, including zero, can be represented in several ways depending on the numerical system and format. Below are some common methods to represent them:

1. Binary Representation:

Fixed-point numbers are represented in binary format where each bit corresponds to a power of 2.

Example:

0->0000

1->0001

15->1111

The number of bits used depends on the range of representation required.

2. Decimal Representation:

The fixed-point integers are represented as decimal numbers directly.

Example: 0,1,2,3,.....,9

3. Hexadecimal Representation:

Fixed-point integers can also be represented in base 16 (hexadecimal), which is more compact than binary.

Example:

0->0

10->A

255->FF

Cont.....

4. Octal Representation:

Numbers are represented in base 8 (octal).

Example:

0->0

7->7

8->10

5. BCD (Binary-Coded Decimal):

In BCD, each decimal digit is stored as a separate binary number.

Example:

5->0101

10-> 0001 0000

27-> 0010 0111

6. Signed Representation (Including Zero):

Although only positive integers and zero are considered here, signed formats can also represent them (e.g., sign-magnitude or two's complement):

Example:

Sign-Magnitude Format: 5-> 0 0101

Two's Complement Format: 0->0000

7. Fixed-Point Format with an Exponent:

In fixed-point arithmetic, integers can represent fixed-point numbers, depending on the exponent.

Example:

Decimal: Exponent=-1 then 0->0.0, ->0.5

Binary: Exponent=-3 then 1->0.010

GTU Questions

Q.9 Define RTL. Give an example of register transfer of data through accumulator.

- Example: Addition of Two Numbers Using an Accumulator

Operation: Add the content of register R1 to the accumulator(AC) and store the result back in the accumulator(AC) .

RTL: $AC \leftarrow AC + R1$

GTU Questions

Q.12 Perform $A - B$ (subtract) operation for the following numbers using signed magnitude number format. (Write necessary assumptions if required) $A = +11$ and $B = -6$.

- Assume we are using 5 bits (1 sign bit + 4 magnitude bits) to represent the numbers:
Sign bit: 0 for positive and 1 for negative
Magnitude bits: Represent the absolute value of the number.

- $A = +11$ (0 1011) $B = -6$ (1 0110)
- Now, Add the magnitudes of A and B, keeping the sign bit separate.

$$A - B = +11 - (-6) = +11 + 6$$

$$1011 + 0110 = 10001$$

- The result of the addition is 10001

Sign bit: 0(positive)

Magnitude: 10001

GTU Questions

Q.14 What is binary and decimal equivalent of F8 hexadecimal value?

1. Hexadecimal to Binary Conversion

Each hexadecimal digit represents 4 bits in binary:

F=1111(binary for 15 in decimal)

8=1000(binary for 8 in decimal)

So, the binary equivalent of F8 is:

$$(F8)_{16} = (1111 \ 1000)_2$$

2. Hexadecimal to Decimal Conversion

To convert $(F8)_{16}$ to decimal:

$$F8_{16} = (15 \times 16^1) + (8 \times 16^0)$$

$$F8_{16} = (15 \times 16) + (8 \times 1)$$

$$(F8)_{16} = 240 + 8 = (248)_{10}$$

GTU Questions

Q.19 Explain how $(r-1)$'s complement is calculated. Calculate 9's complement of 546700.

Here $r=10$, $n=6$ and $N= 546700$

9's complement of 546700 is calculated as:

$$=(r^n - 1) - N$$

$$=(10^6-1)-(546700)$$

$$=(1000000-1)-(546700)$$

$$=999999- 546700$$

$$= 453299$$

Thank you!

